VEYRA Security Self-Assessment Report: Permission-Tier Implementation and Known-Risk Register

Athilla Zaidan Zidna Fann
Informatics Engineering
Institut Teknologi Bandung

Jason Edward Salim
Informatics Engineering
Institut Teknologi Bandung
13524034@std.stei.itb.ac.id

Abel Gani
Information Systems and Technology
Institut Teknologi Bandung
18224016@std.stei.itb.ac.id

Gregory William Sutjipto
Information Systems and Technology
Institut Teknologi Bandung
18224010@std.stei.itb.ac.id

Tiffany Viriya Chu
Information Systems and Technology
Institut Teknologi Bandung
18224068@std.stei.itb.ac.id

Abstract

This report is the security self-assessment of **VEYRA**, a Digital Livestock Passport platform whose AI agent prepares insurance applications, premium payments, and claims that execute only after a manager approves them. It documents how the permission tiers are implemented in code and lists the known risks with their current mitigations and planned remediation. The assessment covers the backend (61 endpoints in 19 route modules), the web client, the camera worker interface, the language-model adapter, and the sandbox insurer and payment providers, at commit `bc7e56c` of the `main` branch. Four human roles and one machine identity are enforced on every route, with separation of duties between preparing and approving a financial action. Every farm-scoped query is isolated by farm, financial actions are bound to a hashed payload, time-limited, and idempotent, and the language model has no tools and cannot decide or execute anything. These controls are verified by automated tests within the 551-test backend suite. The register lists 16 known risks: three rated **High**, all of which arise from prototype-grade identity and secrets (static demo tokens, a default service token, demo mode enabled by default) and are scheduled for closure before any pilot handles real farm data.

1 Scope and Method

1.1 Scope

The assessment covers the components in Table 1. All financial providers are sandboxes: no real money moves, no policy is bound, and no claim is filed with a real insurer. The assessment therefore distinguishes risks that are acceptable in the current demonstration from risks that must be closed before a pilot with real farms, insurers, or payments.

Table 1: Components in scope.

Component	Description
API service	FastAPI, 61 endpoints in 19 route modules, SQLAlchemy 2 persistence, one Docker image
Web client	React 19 + TypeScript progressive web app, served by the API on the same origin
Camera worker interface	Observation feed and gallery synchronisation authenticated by a service token
Perception adapters	Muzzle matcher (ONNX), weight measurement, view selection, tracking
Language-model adapter	GPT-6 Luna for explanations, question parsing, and agent phrasing (structured outputs)
Sandbox providers	Simulated insurer and payment provider behind provider adapters
Storage	Relational database and media files on a local data volume

1.2 Method

The assessment combined four activities. (1) A code review of every route's authentication dependency, every role capability, and every farm-scoped query. (2) A review of the financial action lifecycle, the media and upload paths, and the language-model boundary. (3) A lightweight threat model of each trust boundary using the STRIDE categories (spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege). (4) Verification against the automated test suite, whose security-relevant tests are listed in Appendix A. Each risk is rated by likelihood and impact on a three-point scale, with the rating given by the higher of the two when either is high.

2 System and Trust Boundaries

Figure 1 shows the architecture. Table 2 lists the trust boundaries and the control that authenticates each crossing.

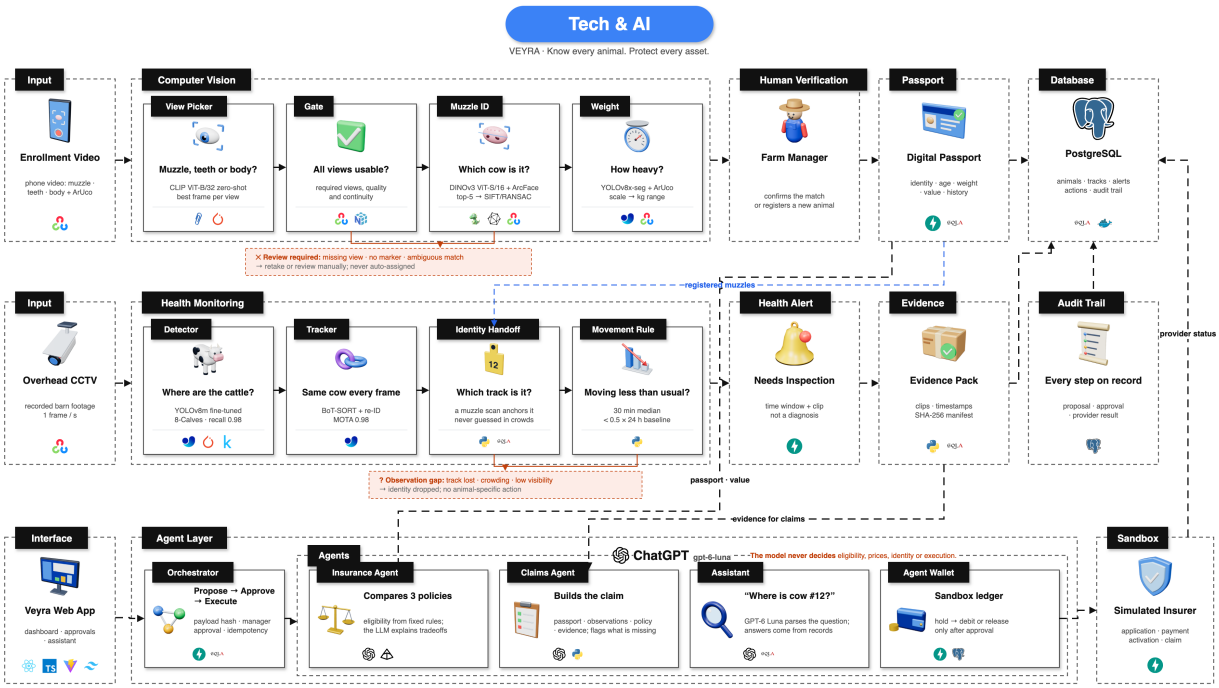


Figure 1: VEYRA architecture. Every human request crosses the API with a bearer token; the camera worker uses a service token; the agent wallet only executes manager-approved payloads.

Table 2: Trust boundaries and their authentication.

Boundary	Credential	Control
Browser → API	Bearer token per user	Constant-time comparison; role and farm resolved server-side
Camera worker → API	`X-Service-Token`	Constant-time comparison; observation feed and gallery acknowledgement only
Browser → media	Signed URL (`exp`, `sig`)	HMAC-SHA256, one-hour expiry, uniform `NOT_FOUND`
API → language model	Provider API key (environment)	Text only, structured outputs, no tools
API → sandbox providers	In-process adapter	Idempotency key per execution; provider record reconciles retries
API → storage	Process filesystem	Data volume outside the image; media addressed by id, never by path

3 Permission-Tier Implementation

3.1 Authentication

A person presents `Authorization: Bearer <token>`. The API looks the token up in its own short read session, so authentication never holds a write lock, and compares it with `hmac.compare_digest` to avoid timing side channels (`domain/auth.py`). The result is an immutable `Principal` carrying

the user id, farm id, role, and name; the client never supplies its own role or farm. The camera worker presents `X-Service-Token`, compared the same way, and receives a `ServicePrincipal` that holds no farm and no human role. A missing or invalid credential always returns the same `UNAUTHENTICATED` error envelope, and an empty configured service token is rejected rather than treated as a match.

3.2 Role Tiers

Roles are declared once as capability tuples, so each route reads as a rule (Table 3).

Table 3: Permission tiers and their capability sets.

Tier	Capability sets	Authority
1 · Manager	`APPROVERS`, `ENROLLERS`, `ALERT_REVIEWERS`	The only role that approves, declines, executes, or reconciles a financial action; confirms claim declarations; publishes the camera gallery; resets the demo
2 · Operator	`ENROLLERS`, `ALERT_REVIEWERS`	Enrolls animals, uploads media, confirms valuations, prepares actions, drafts claims, reviews tracks and alerts; cannot approve
3 · Veterinarian	`ALERT_REVIEWERS`, `FINDING_AUTHORS`	The only role that adds professional findings to an incident; no payment authority
4 · Assessor	Read only	Reads sandbox submissions; cannot change any record
Machine · Service worker	Service token	Posts camera observations and acknowledges gallery versions; no human capability
AI agent	None (no principal)	Prepares actions inside a manager's or operator's session; never approves or executes

3.3 Permission Matrix

Table 4 maps endpoint groups to tiers, as implemented by the route dependencies (`require_roles`, `service_principal`, `service_or_roles`).

Table 4: Permission matrix (M = manager, O = operator, V = veterinarian, A = assessor, S = service worker).

Operation (endpoint group)	M	O	V	A	S
Read herd, passports, policies, alerts, claims, audit, wallet	**Yes**	**Yes**	**Yes**	**Yes**	–
Ask the assistant	**Yes**	**Yes**	**Yes**	**Yes**	–
Create, upload, process, confirm, retry enrollments	**Yes**	**Yes**	–	–	–
Confirm valuations; request insurance recommendations	**Yes**	**Yes**	–	–	–
Propose, revise, cancel agent actions; converse with the agent	**Yes**	**Yes**	–	–	–
Approve or decline, execute, reconcile agent actions	**Yes**	–	–	–	–
Record incidents, draft claims, run claim identity checks	**Yes**	**Yes**	–	–	–
Update a claim (declaration)	**Yes**	–	–	–	–
Add professional findings	–	–	**Yes**	–	–
Review alerts	**Yes**	**Yes**	**Yes**	–	–
Assign tracks; start demo replays	**Yes**	**Yes**	–	–	–
Policy activation check; publish camera gallery	**Yes**	–	–	–	–
Post camera observations; acknowledge gallery	–	–	–	–	**Yes**
Read a camera gallery	**Yes**	–	–	–	**Yes**
Reset the demo (demo farm and demo mode only); change sandbox settings	**Yes**	–	–	–	–

Three routes take no bearer token by design: `GET /health`, `GET /media/{id}` (the HMAC signature is the credential), and `GET /demo/footage/{name}`, which streams only three named, AI-generated demonstration clips that contain no farm data and exists only in demo mode.

3.4 Tenant Isolation

Every farm-scoped query filters on `Principal.farm_id`. A record of another farm is reported as `NOT_FOUND`, never as forbidden, so a caller cannot even learn that it exists. The same rule protects the camera pipeline: a muzzle match against an animal of another farm is recorded as `candidate_not_registered`, so a camera never learns that another farm's animal exists. The demo reset additionally requires a manager of the demo farm, so a manager of another farm cannot reset it.

3.5 Approval-Bound Financial Actions

Financial integrity rests on the action lifecycle rather than on the user interface:

- **Server-built payloads.** The server builds every payload from current records; the client never supplies an amount, recipient, or term.
- **Hash binding.** The payload is fixed by $h = \text{SHA256}(\text{canonical_json}(\text{payload}))$, and an approval must name the exact hash and version that the manager reviewed.
- **Separation of duties.** Only a manager approves, and an operator cannot approve an action even when the operator prepared it.
- **Expiry and staleness.** An action expires 30 minutes after it is proposed. Before execution the payload is rebuilt from current records; any change to amount, recipient, terms, animal, or evidence cancels the approval as `approval_stale`.

- **Idempotent execution.** Execution requires an `Idempotency-Key`; a key reused for another action, or a new key after success, is rejected as a duplicate. Execution runs as three short transactions with a balance hold, so a timeout keeps the hold and a retry reconciles from the provider's record instead of paying twice.
- **Non-repudiation.** The agent's proposal, the manager's decision, and the provider's result are three separate audit events tied to one animal. The audit module only inserts events; it exposes no update or delete path.

3.6 AI Agent Containment

The language model is confined to language. It has no tools and cannot call the API. Every call uses a typed output schema, and every system prompt ends with an instruction to treat the farm user's text as data and never as instructions. Eligibility, prices, identity, and execution are computed by deterministic code; explanations are checked against the inputs, and items for products that were not supplied or terms references that do not exist are dropped before display. The in-app agent prepares actions but never approves them: its action-review card points to a pending action that only a manager can stamp. If the model is unavailable, refuses, or returns unparseable output, a deterministic template states the same facts and is labelled as such.

3.7 Media and Upload Controls

- **Signed downloads.** Media URLs carry `exp` and $sig = \text{HMAC-SHA256}(secret, \texttt{id:exp})$ with a one-hour lifetime. A bad, tampered, or expired signature, an unknown id, and a missing file all return the same `NOT_FOUND`, so ids cannot be probed.
- **Safe serving.** Responses set `X-Content-Type-Options: nosniff`, and documents download as attachments instead of rendering in the application's origin.
- **Type allow-lists.** Videos, photos, and documents are accepted only from explicit content-type lists; documents are also checked by magic bytes (PDF, JPEG, PNG), so a renamed file is refused.
- **Size caps.** 200 MiB for enrollment videos, 50 MiB for claim identity scans, and 10 MiB for documents, enforced while streaming and refused early from the `Content-Length` header.
- **Integrity.** Every stored file is hashed with SHA-256 as it is written, and evidence manifests cite those hashes.

3.8 Identity Assurance Controls

Because the platform's value depends on the right animal being insured and claimed, identity is treated as a security control. A muzzle match must exceed five geometrically verified points with crop quality of at least 0.6, a crowded scan zone identifies nobody, and an identity released by the continuity policy is never re-attached automatically. Every death claim requires a fresh muzzle scan that is matched against the claimed animal *and* every other confirmed animal of the farm; a mismatch names the other animal and blocks submission, and a newer check after approval makes the approved submission stale.

4 Known-Risk Register

Table 5 lists the known risks. Ratings describe exposure in a pilot or production deployment; in the current sandbox demonstration no real funds or real farm records are at stake.

Table 5: Known-risk register.

ID	Risk	Rating	Current mitigation	Planned remediation
R1	Static, guessable demo bearer tokens; no login, session expiry, or MFA; tokens stored in plain text	High	Tokens compared in constant time; tokens resolve role and farm server-side; demo-only users	OIDC/SSO login, short-lived signed sessions, hashed tokens, MFA for managers
R2	Default service token (`demo-worker-token`) when `VEYRA_SERVICE_TOKEN` is unset	High	Service tier limited to observations and gallery acknowledgement	Refuse to start outside demo mode without a secret; per-camera credentials or mTLS
R3	Demo mode on by default: reset, sandbox switches, replays, and public footage route exposed	High	Reset requires a manager of the demo farm; footage route serves three fixed clips without farm data	Default `VEYRA_DEMO_MODE=false` in production builds; deployment check
R4	Farm-wide read access for every role, including the assessor (wallet, audit, all passports)	Medium	Reads stay scoped to the user's farm; all writes are role-gated	Per-role read scopes; assessor limited to submitted claims and their evidence
R5	No rate limiting or brute-force protection on authentication, uploads, and assistant queries	Medium	Upload size caps; uniform errors reveal nothing to enumerate	Gateway rate limits per token and IP; per-farm quotas on model calls
R6	Transport security delegated: the container serves HTTP on port 8000; no CSP or frame headers	Medium	Same-origin web app; `nosniff` on media; documents as attachments	TLS termination with HSTS; Content-Security-Policy, `X-Frame-Options`, Referrer-Policy
R7	Data at rest unencrypted on a local volume (database and media), no backup policy	Medium	Data volume outside the image; media addressed by id; SHA-256 per file	Managed PostgreSQL with encryption at rest, backups, and point-in-time recovery; encrypted object storage
R8	Audit trail append-only in application code but not tamper-evident in storage	Medium	No update or delete path in the audit module; separate proposal, decision, and result events	Hash-chained audit events and write-once storage for the audit log
R9	Presentation attack on the muzzle biometric (printed or replayed muzzle of another animal)	Medium	Continuous guided video; human confirmation; claim-time re-scan against every confirmed animal	Liveness and presentation-attack detection on the enrollment and claim scans
R10	Wrong identity association on CCTV in barns unlike the tuning footage	Low	`handoff_v2` releases doubtful identities (zero wrong identities on real footage); crowded scans identify nobody	Revalidate on each farm's footage before enabling alerts; cattle-specific re-identification

ID	Risk	Rating	Current mitigation	Planned remediation
R11	Prompt injection through farm text or documents; farm data sent to a third-party model provider	Medium	No tools; typed outputs; grounding check; text treated as data; deterministic fallback	Data-processing agreement, data minimisation (no personal data in prompts), regional or self-hosted model option
R12	Personal data obligations (farmer, staff, and veterinarian names) under Indonesia's Personal Data Protection Law (UU No. 27/2022)	Medium	Minimal personal data; role-based access; no sale or sharing	Privacy notice and consent, retention policy, data-subject request process
R13	Availability: background jobs and camera replays run in one API process	Low	Single worker by design; three-step execution survives restarts via reconciliation	Separate job workers and queue; health checks and autoscaling
R14	Uploaded files are not malware-scanned	Low	Type allow-lists, magic bytes, size caps; files never rendered in the app origin	Antivirus scanning in the upload pipeline
R15	Media links keep working until expiry once shared; random per-process media secret if unset	Low	One-hour expiry; HMAC signature; uniform `NOT_FOUND`	Shorter lifetimes for documents; require a configured secret and key rotation
R16	Third-party licences and supply chain (Ultralytics AGPL-3.0, DINOv3 licence, CDN-free build)	Low	Locked dependencies (`uv.lock`, `pnpm-lock.yaml`); licences recorded in the model cards	Licence review before closed-source distribution; dependency scanning in CI

5 Remediation Roadmap

The risks are scheduled against the product roadmap so that each control is in place before the exposure it addresses becomes real.

Table 6: Remediation plan by phase.

Phase	Risks closed	Exit criterion
Before a pilot with real farm data	R1, R2, R3, R6	Real login with MFA for managers, no default secrets, demo mode off, TLS and security headers
During the identity and monitoring pilot	R4, R5, R7, R11, R12	Least-privilege reads, rate limits, encrypted managed database with backups, data-processing agreement, privacy notice
Before connecting a real insurer or payment provider	R8, R9, R14, R15	Tamper-evident audit, presentation-attack detection, malware scanning, external penetration test
At scale	R10, R13, R16	Per-farm revalidation of tracking, separate job workers, dependency and licence scanning in CI

6 Conclusion

VEYRA's core security property, that no money moves and no claim is submitted without a manager approving exactly what will happen, is enforced in the backend by role tiers, separation of duties, hash-bound and time-limited approvals, idempotent execution, and an audit trail that separates agent, human, and provider. Tenant isolation, signed media, upload validation, and a language model without tools complete the defence. The highest risks are prototype-grade identity and secrets, which are expected for a sandbox demonstration and are the first items on the remediation plan before any real farm, insurer, or payment provider is connected.

A Appendix A: Security Tests

The following automated tests from the backend suite verify the controls described in this report.

Control	Tests
Authentication	`test_missing_token_is_unauthenticated`, `test_bad_token_is_unauthenticated`, `test_read_routes_require_a_token`, `test_service_token_accepted`, `test_service_token_rejected`, `test_authenticate_service_rejects_empty_configured_token`, `test_observation_feed_requires_the_service_token`, `test_gallery_needs_the_service_token_or_a_manager`
Role tiers and separation of duties	`test_operator_cannot_approve`, `test_operator_cannot_approve_what_the_agent_prepared`, `test_operator_decision_is_forbidden_even_when_operator_proposed`, `test_operator_cannot_run_the_activation_check`, `test_operator_cannot_reset`, `test_route_rejects_vet_and_other_farm`
Tenant isolation	`test_cross_farm_action_access_is_not_found`, `test_other_farm_cannot_see_passport`, `test_other_farm_animal_is_not_found`, `test_other_farm_cannot_see_or_review_the_alert`, `test_other_farm_sees_only_its_own_herd_and_audit`, `test_other_farm_events_are_not_in_the_passbook`, `test_other_farms_see_no_scan_zone_of_ours`, `test_other_farm_manager_cannot_reset_the_demo`, `test_unregistered_or_other_farm_candidate_leaves_the_track_unassigned`
Financial integrity	`test_charge_is_idempotent_per_key`, `test_duplicate_execution_is_rejected`, `test_key_reused_for_another_action_is_duplicate`, `test_new_key_after_success_is_duplicate`, `test_declined_action_cannot_execute`, `test_execution_blocked_after_expired_approval`, `test_execution_blocked_after_stale_approval`, `test_new_valuation_after_approving_payment_is_stale`, `test_payment_decline_fails_and_releases_hold`
Media and uploads	`test_tampered_signature_is_not_found`, `test_expired_signature_is_not_found`, `test_unknown_media_is_not_found`, `test_verify_signature_rejects_empty_and_expired`, `test_store_rejects_content_type_outside_allow_list`, `test_oversize_upload_is_refused_from_its_content_length`
Identity assurance	`test_crowded_scan_identifies_nobody_and_raises_no_animal_alert`, `test_an_empty_zone_identifies_nobody`, `test_sample_mismatch_blocks_submission_with_the_reason`, `test_later_mismatch_makes_an_approved_submission_stale`, `test_no_enrolled_image_is_uncertain_not_a_mismatch`

B Appendix B: Deployment Hardening Checklist

Setting	Production value
`VEYRA_DEMO_MODE`	`false`: hides reset, replays, sandbox switches, and the footage route
`VEYRA_SERVICE_TOKEN`	A long random secret per deployment, rotated; never the default
`VEYRA_MEDIA_SECRET`	A long random secret, so signed links survive restarts and keys can be rotated
`VEYRA_DATABASE_URL`	A managed, encrypted database with backups
Model provider key	Stored in the platform's secret manager, never in the image or repository
Reverse proxy	TLS, HSTS, security headers, request size limits, and rate limits

C Appendix C: Product Links

GitHub repository: `https://github.com/ShibainuID/Veyra`